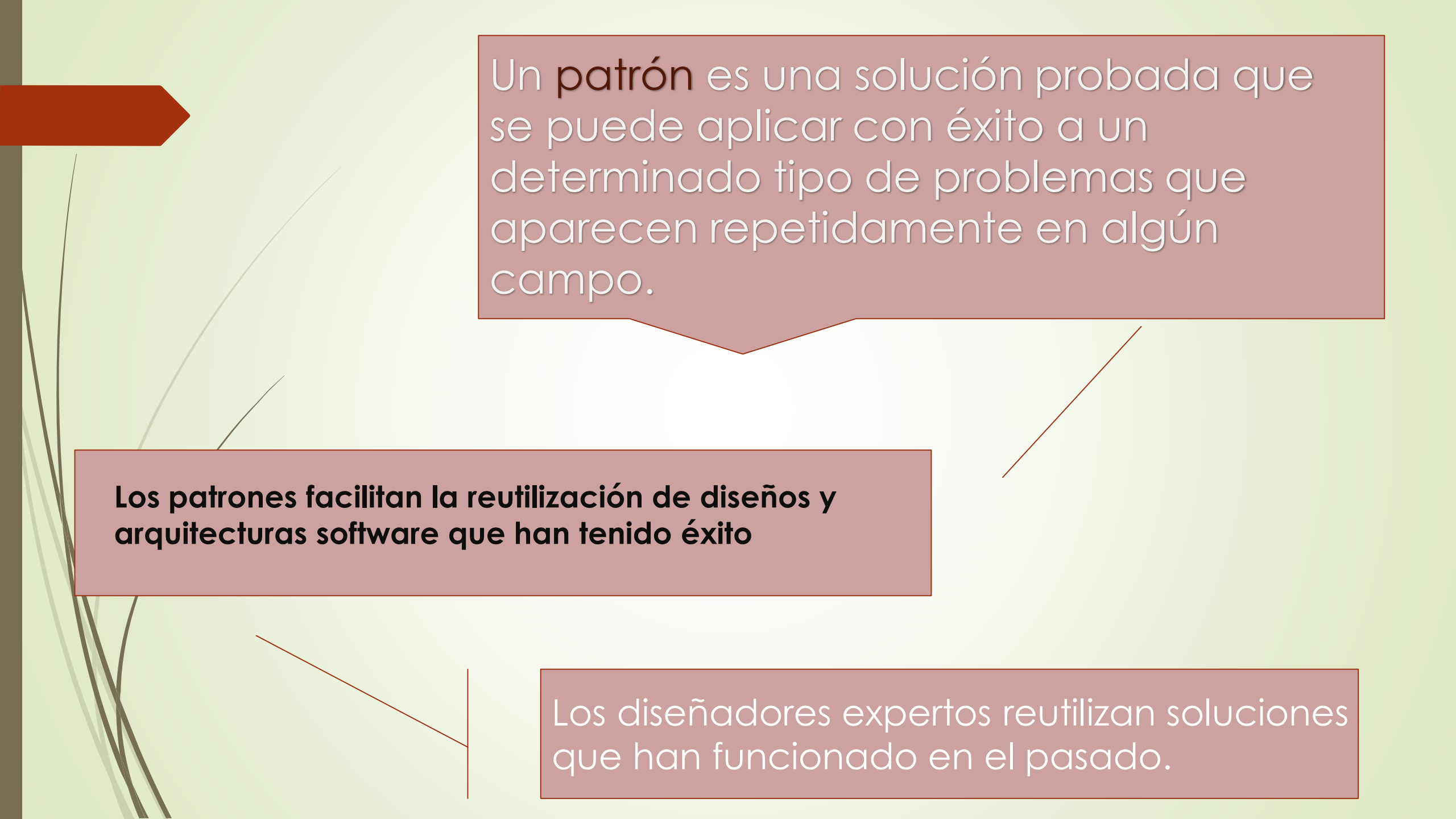


Patrones de Diseño

Ingeniería del Software II



Un **patrón** es una solución probada que se puede aplicar con éxito a un determinado tipo de problemas que aparecen repetidamente en algún campo.

Los patrones facilitan la reutilización de diseños y arquitecturas software que han tenido éxito

Los diseñadores expertos reutilizan soluciones que han funcionado en el pasado.



¿Por qué son útiles los patrones de diseño?



Te ahorran tiempo

Buscar siempre una nueva solución a los mismos problemas reduce tu eficacia como desarrollador.



Te ayudan a estar seguro de la validez de tu código

Los patrones de diseño son estructuras probadas por millones de desarrolladores a lo largo de muchos años.



Establecen un lenguaje común

Los patrones de diseño establecen un lenguaje común entre todos los miembros del equipo.



Elementos de un Patrón de Diseño

- 
- NOMBRE
 - PROBLEMA
 - SOLUCION
 - CONSECUENCIAS

Clasificación

Arquitecturales	De diseño	Elementales
Expresan un paradigma fundamental para estructurar un sistema software	Compuestos de varias unidades arquitecturales más pequeñas. Describen el esquema básico para estructurar subsistemas y componentes	Específicos de un lenguaje de programación

Clasificación

según el propósito para el que han sido definidos:

CREACIONALES

encapsular y abstraer dicha creación.
inicialización y configuración de clases y objetos
facilitan la tarea de creación de nuevos objetos.

ESTRUCTURALES

especifican la forma en la que unas clases se relacionan con otras.

¿Cómo se componen clases y objetos?

DE COMPORTAMIENTO

gestionan algoritmos, relaciones y responsabilidades entre objetos.

¿Cómo interaccionan y se distribuyen responsabilidades los objetos?

Clasificación

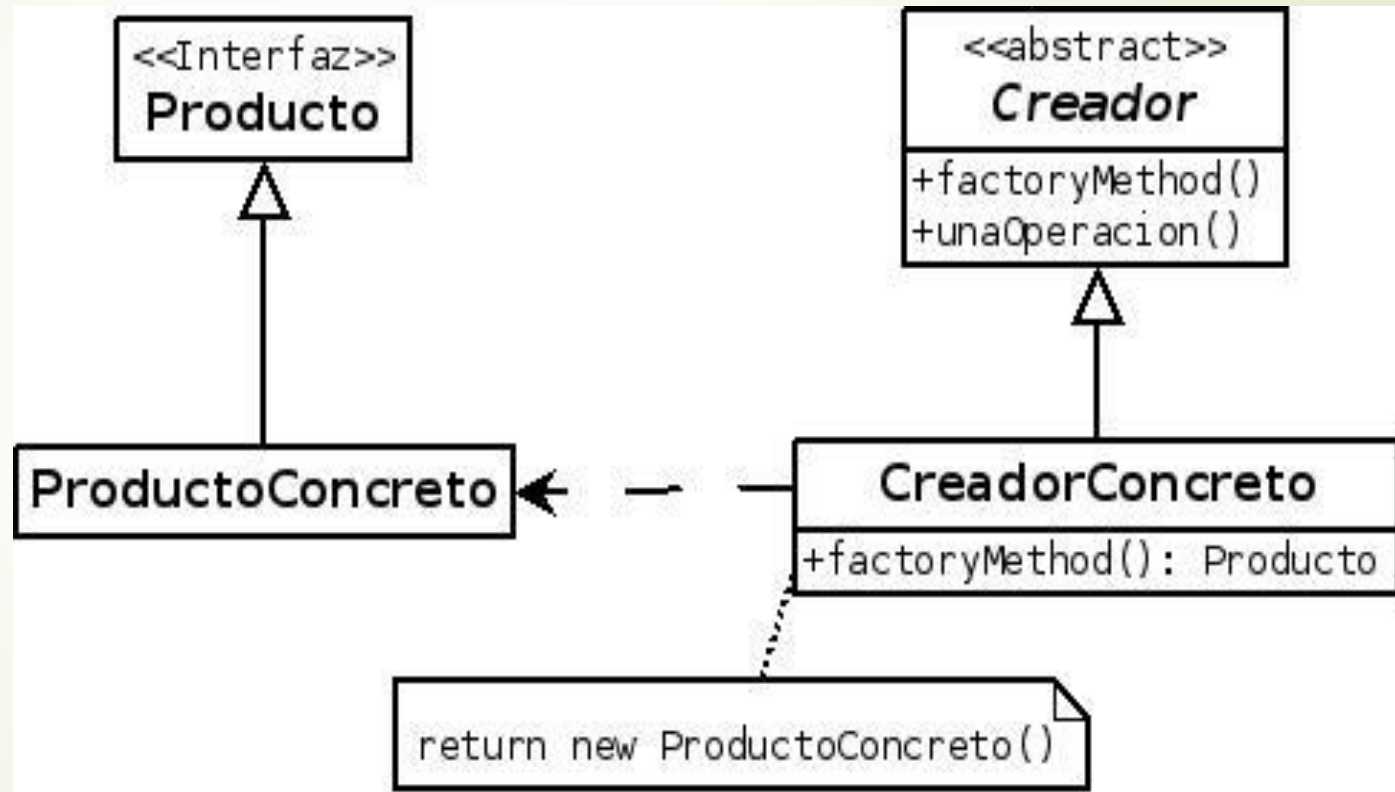
Creacional	Fábrica Abstracta, Método Fábrica, Constructor, Prototipo, Singular
Estructural	Adaptador, Puente, Compuesto, Decorador, Fachada, Peso Mosca, Apoderado
De Comportamiento	Cadena de Responsabilidad, Comando, Iterador, Mediador, Memento, Observador, Estado, Estrategia, Visitante, Método Plantilla

METODO FACTORIA

Expone un método de creación, delegando en las subclases la implementación de este método.

Aplicación

- Cuando una clase no puede anticipar el tipo de objetos que debe crear
- Cuando una clase quiere que sus clases especifiquen los objetos que deben crear
- Cuando una clase delega la responsabilidad a una subclase de ayuda (entre varias), y se quiere localizar el conocimiento de qué subclase de ayuda es la delegada

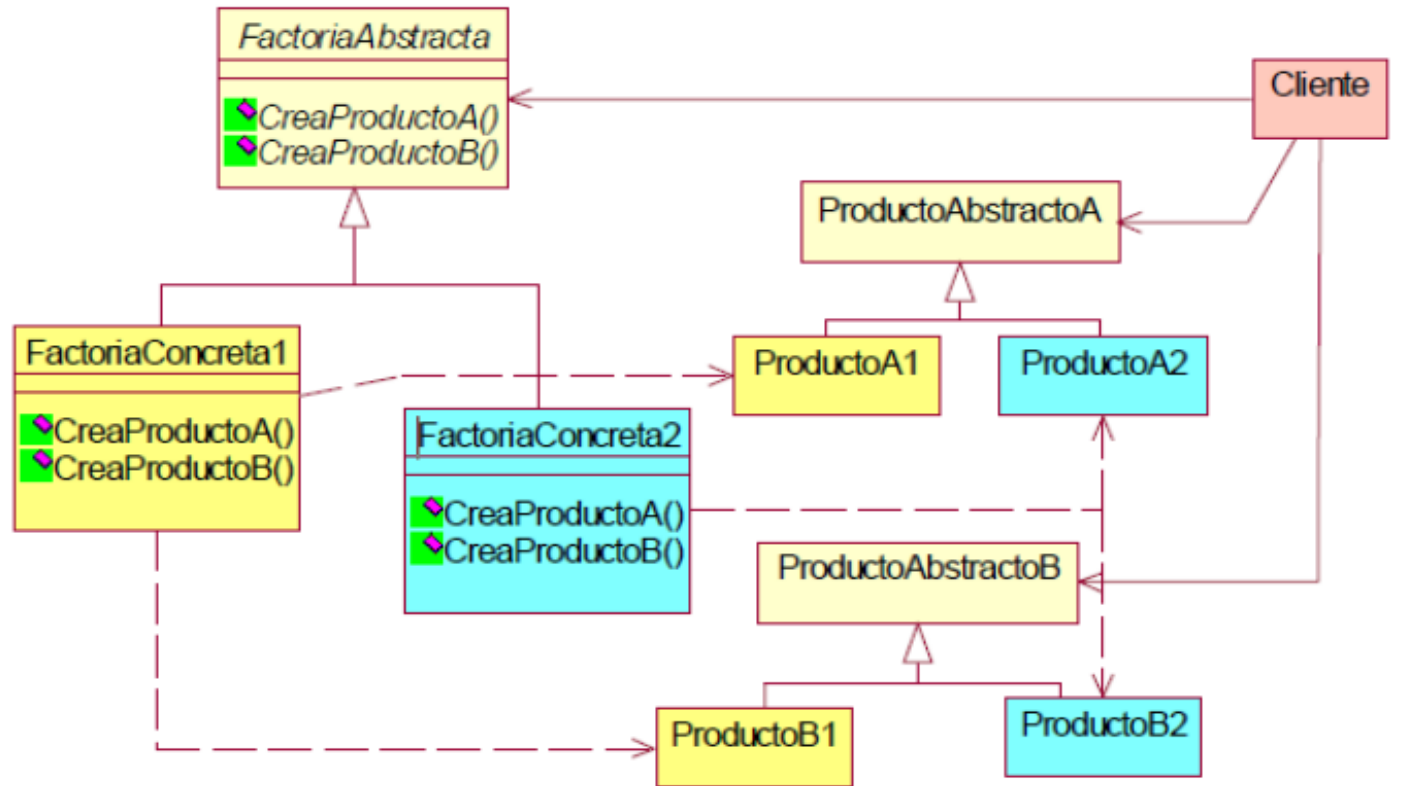


FACTORIA ABSTRACTA

Nos provee una interfaz que delega la creación de un conjunto de objetos relacionados sin necesidad de especificar en ningún momento cuáles son las implementaciones concretas.

Aplicación

- Sistemas independientes de cómo se crean, componen y representan sus productos
- Sistemas que pueden configurarse con una de varias familias de productos
- Obligar a utilizar juntos objetos de una familia de productos
- Ofrecer una librería de clases, mostrando sólo sus interfaces y no sus implementaciones

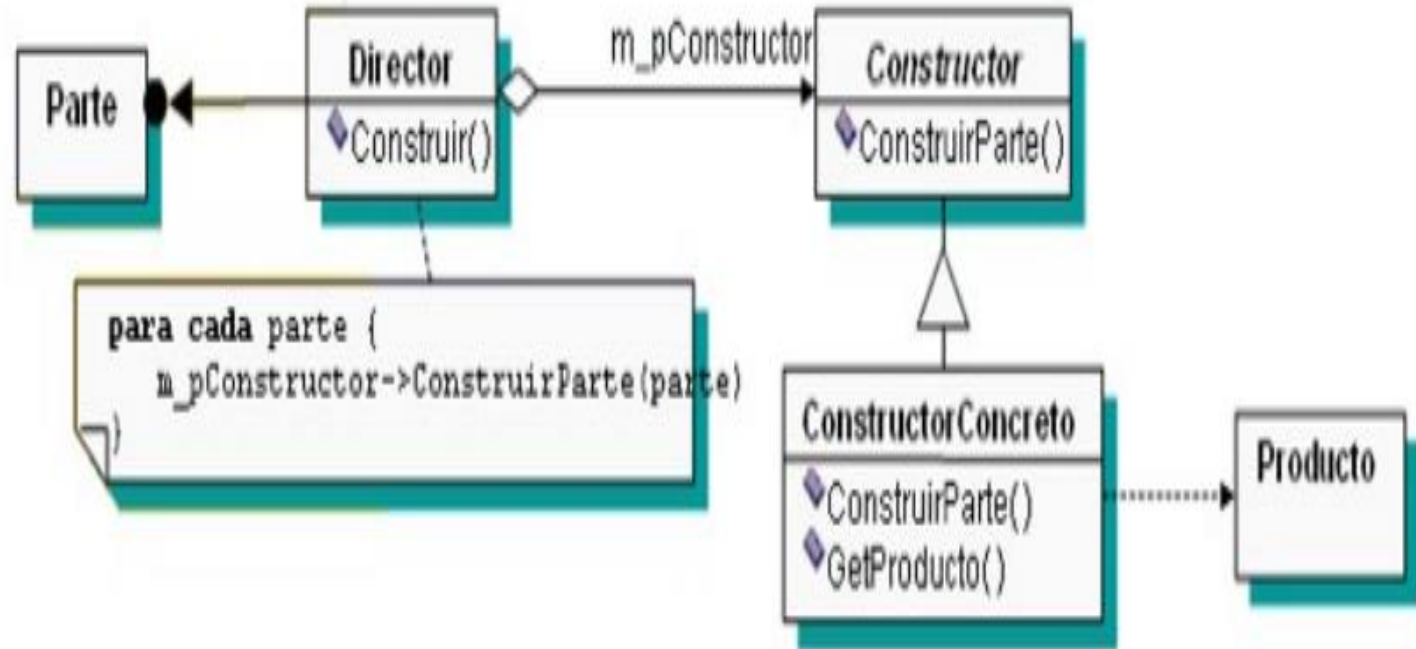


BUILDER

Separa la creación de un objeto complejo de su estructura, de tal forma que el mismo proceso de construcción nos puede servir para crear representaciones diferentes.

Aplicación

- Cuando el algoritmo para crear un objeto complejo debe ser independiente de las partes que constituyen el objeto y cómo se juntan
- Cuando el proceso de construcción debe permitir representaciones diferentes para el objeto que se está construyendo

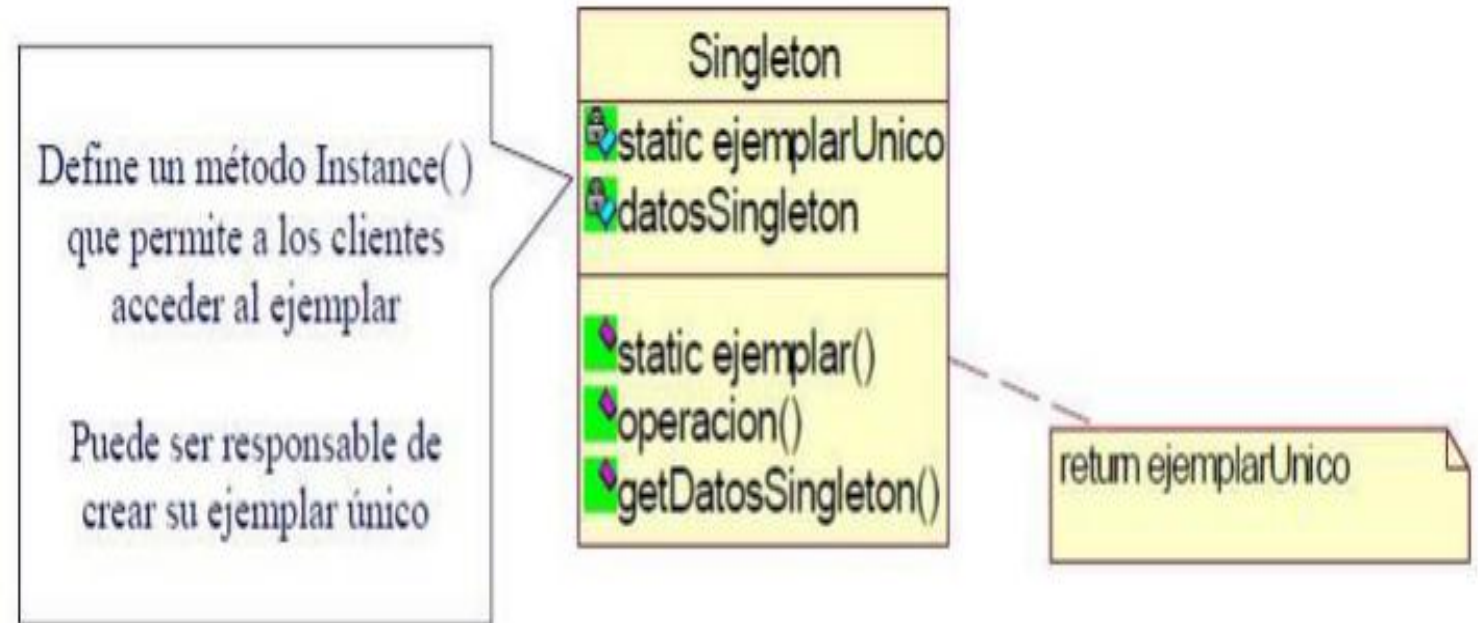


SINGLETON

limita a uno el número de instancias posibles de una clase en nuestro programa, y proporciona un acceso global al mismo.

Aplicación

- Cuando sólo puede haber un ejemplar de una clase, y debe ser accesible a los clientes desde un punto de acceso bien conocido
- Cuando el único ejemplar pudiera ser extensible por herencia, y los clientes deberían usar el ejemplar de una subclase sin modificar su código

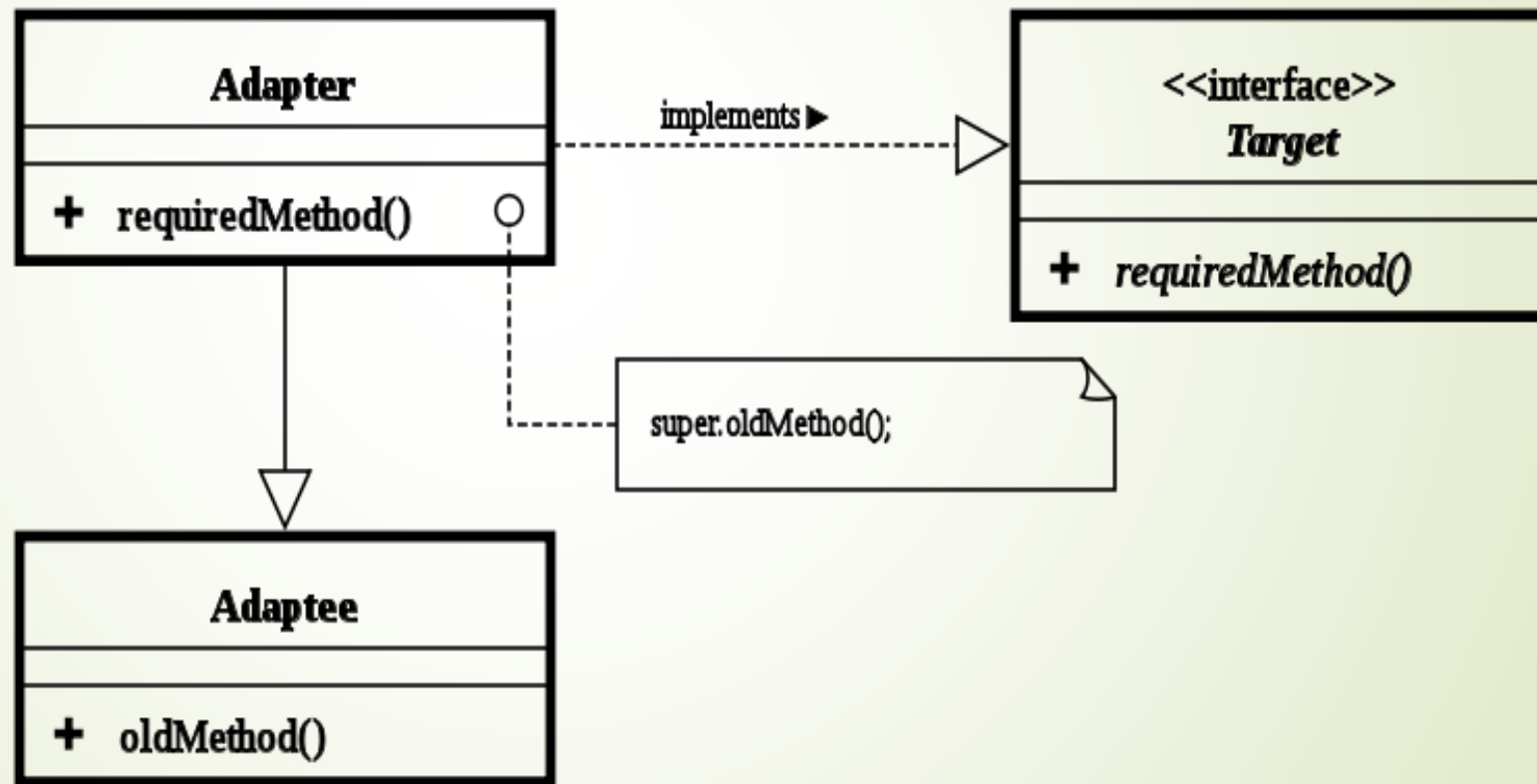


ADAPTADOR

Permite a dos clases con diferentes interfaces trabajar entre ellas, a través de un objeto intermedio con el que se comunican e interactúan.

Aplicación utilizar cuando...

- Se quiera utilizar una clase ya existente y su interfaz no se corresponda con la interfaz que se necesita
- Se quiera envolver código no orientado a objeto con forma de clase

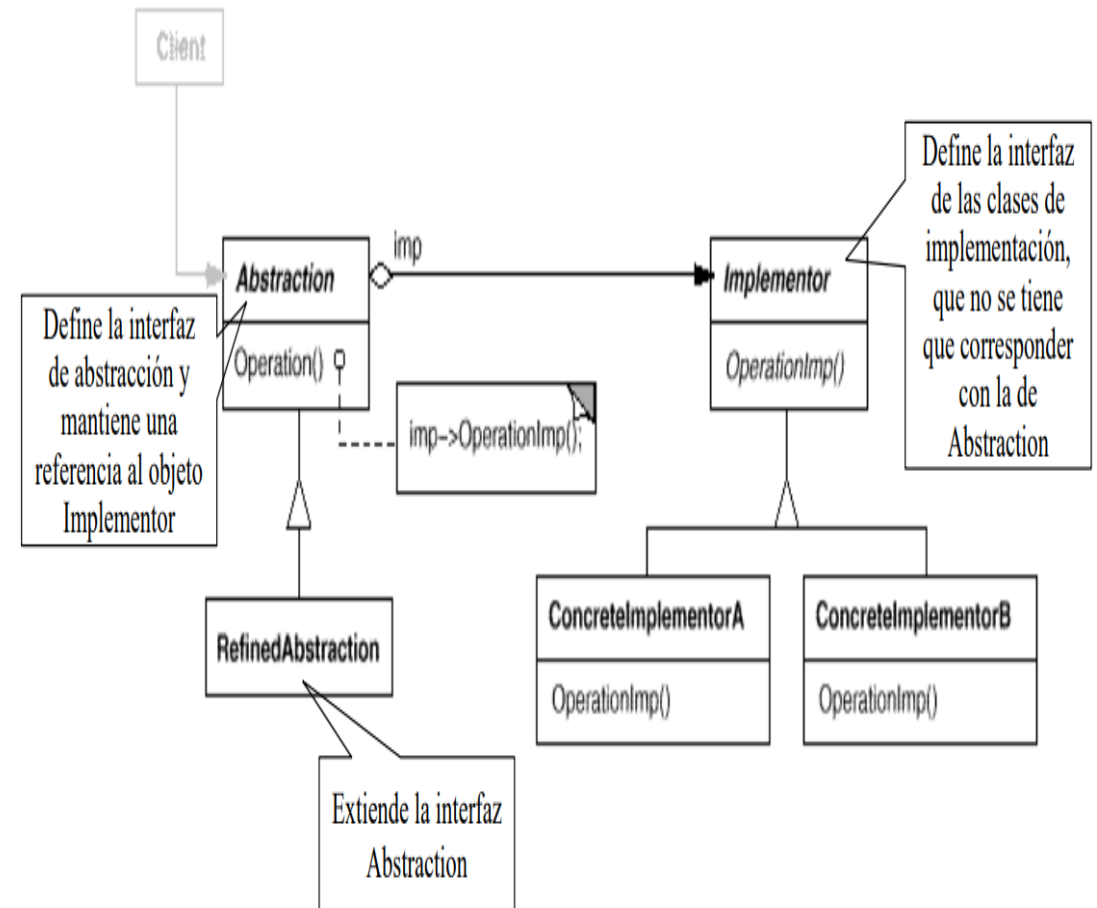


BRIDGE

Desacopla una abstracción de su implementación, para que las dos puedan evolucionar de forma independiente.

Aplicación

- evitar una vinculación permanente entre una abstracción y su implementación
- cambiar la implementación en tiempo de ejecución
- abstracciones e implementaciones podrían ser susceptibles de extensión mediante subclasses
- combinar las abstracciones e implementaciones y extenderlas independientemente
- reducir la proliferación de clases
- que cambios en la implementación de una abstracción no hagan recompilar el código de los clientes
- compartir una implementación entre múltiples objetos, sin que lo noten los clientes

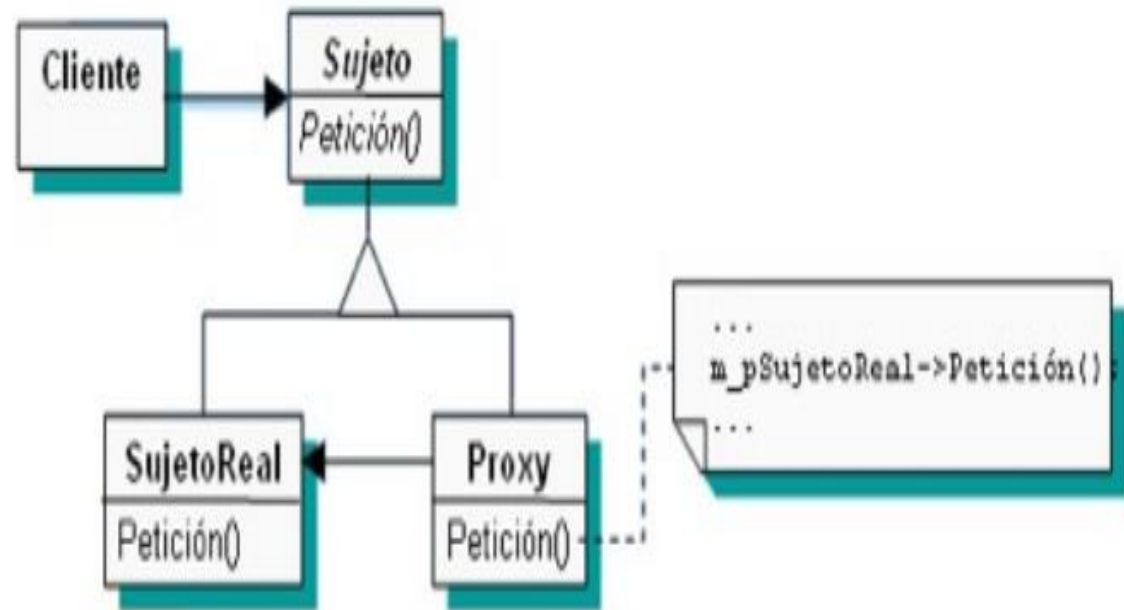


PROXY

Es una clase que funciona como interfaz hacia cualquier otra cosa: una conexión a Internet, un archivo en disco o cualquier otro recurso que sea costoso o imposible de duplicar.

Aplicación

- Se necesite una forma más versátil o sofisticada de referencia a un objeto que un simple puntero. Hay varios tipos de proxy:
- Proxy remoto: es un representante local para un objeto de otro proceso, posiblemente en otra máquina. También se le denomina embajador.
- Proxy virtual: crea objetos costosos bajo demanda.
- Proxy de protección: controla el acceso al objeto original cuando hay distintos niveles de acceso.

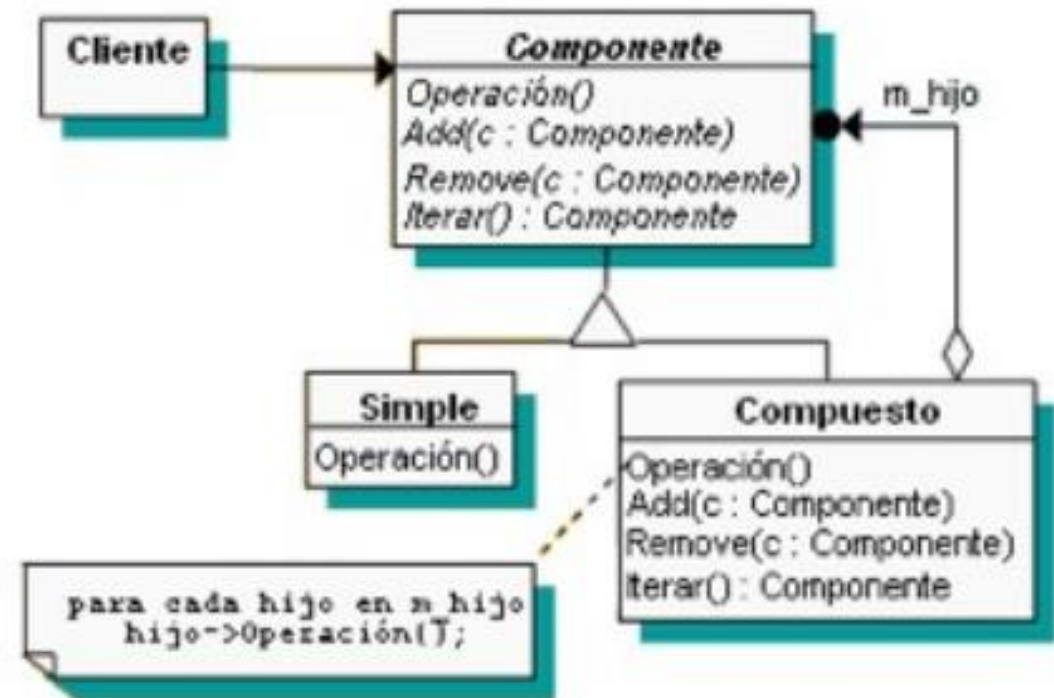


COMPOSITE

Facilita la creación de estructuras de objetos en árbol, donde todos los elementos emplean una misma interfaz. Cada uno de ellos puede a su vez contener un listado de esos objetos, o ser el último de esa rama.

Aplicación

- Se quiera representar jerarquías de objetos todo-parce
- Se quiera que los clientes puedan ignorar la diferencia entre objetos simples y compuestos y tratarlos uniformemente

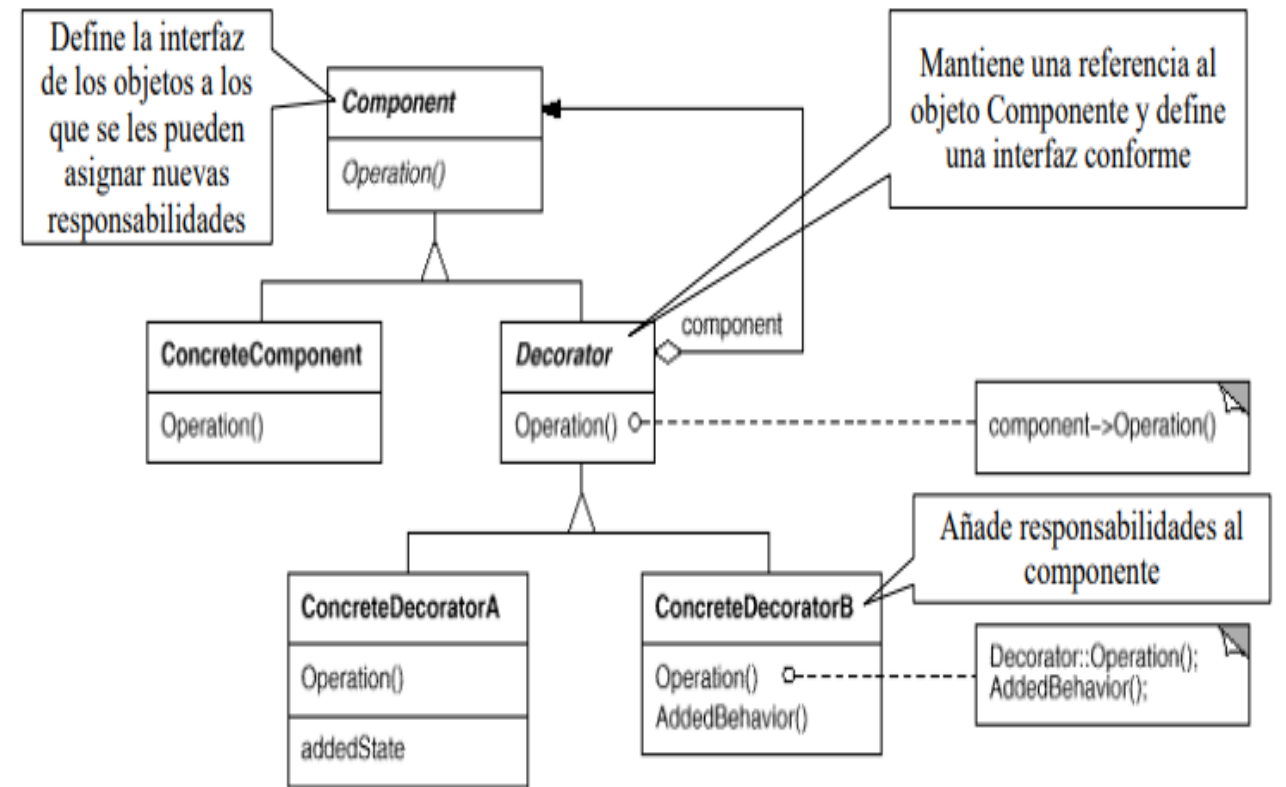


DECORADOR

Permite añadir funcionalidad extra a un objeto (de forma dinámica o estática) sin modificar el comportamiento del resto de objetos del mismo tipo.

Aplicación

- • Para asignar responsabilidades a objetos individuales de forma dinámica y transparente, sin afectar a otros objetos
- • Para responsabilidades que pueden ser suprimidas
- • Cuando no es práctico utilizar herencia de clases (porque se produciría una explosión del número de clases, o porque no se tiene la definición de la clase)

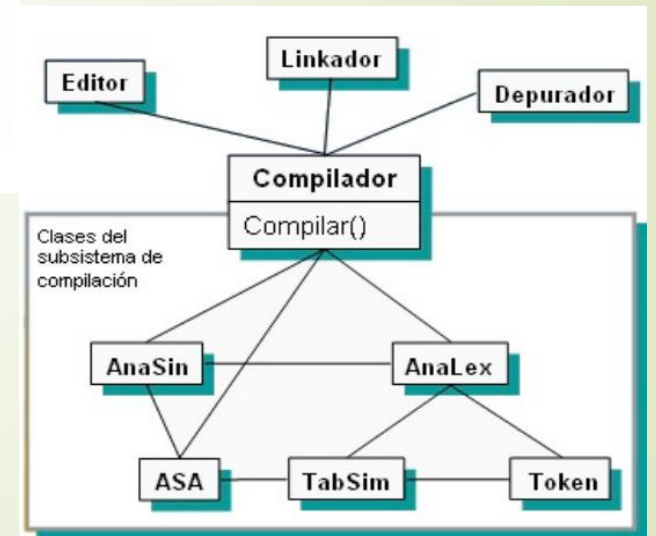
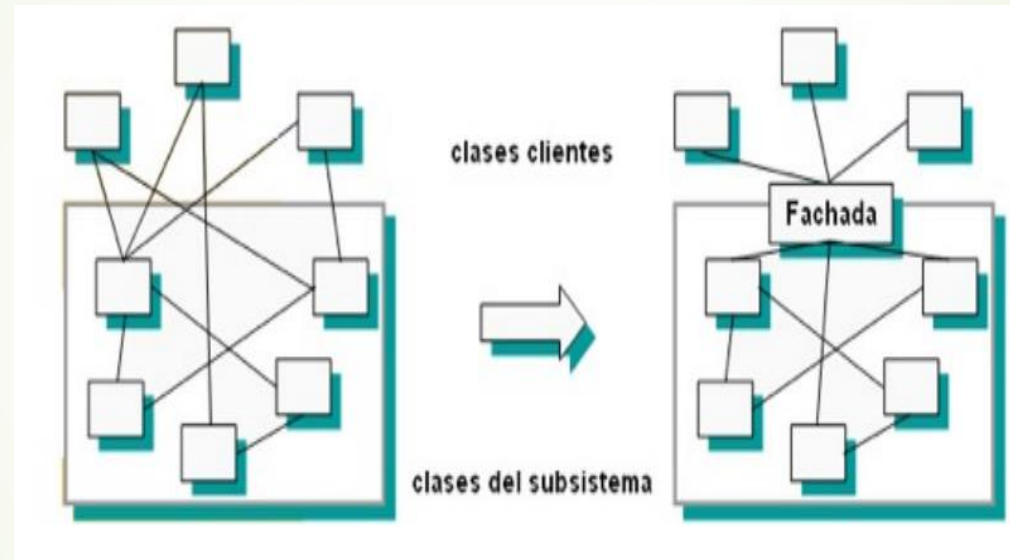


FACHADA

Una fachada es un objeto que crea una interfaz simplificada para tratar con otra parte del código más compleja, de tal forma que simplifica y aísla su uso. Un ejemplo podría ser crear una fachada para tratar con una clase de una librería externa.

Aplicación

- Se quiera proporcionar una interfaz sencilla para un subsistema complejo
- Se quiera desacoplar un subsistema de sus clientes y de otros subsistemas, haciéndolo más independiente y portable
- Se quiera dividir los sistemas en niveles: las fachadas serían el punto de entrada a cada nivel

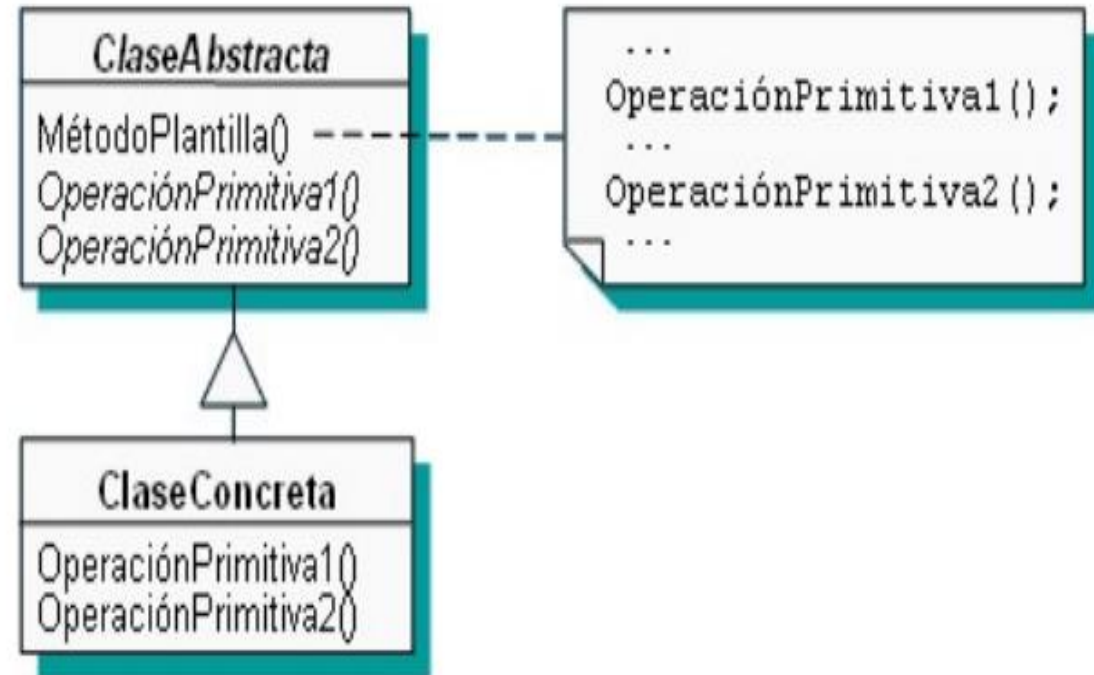


METODO TEMPLATE

Especifica el esqueleto de un algoritmo, permitiendo a las subclases definir cómo implementan el comportamiento real.

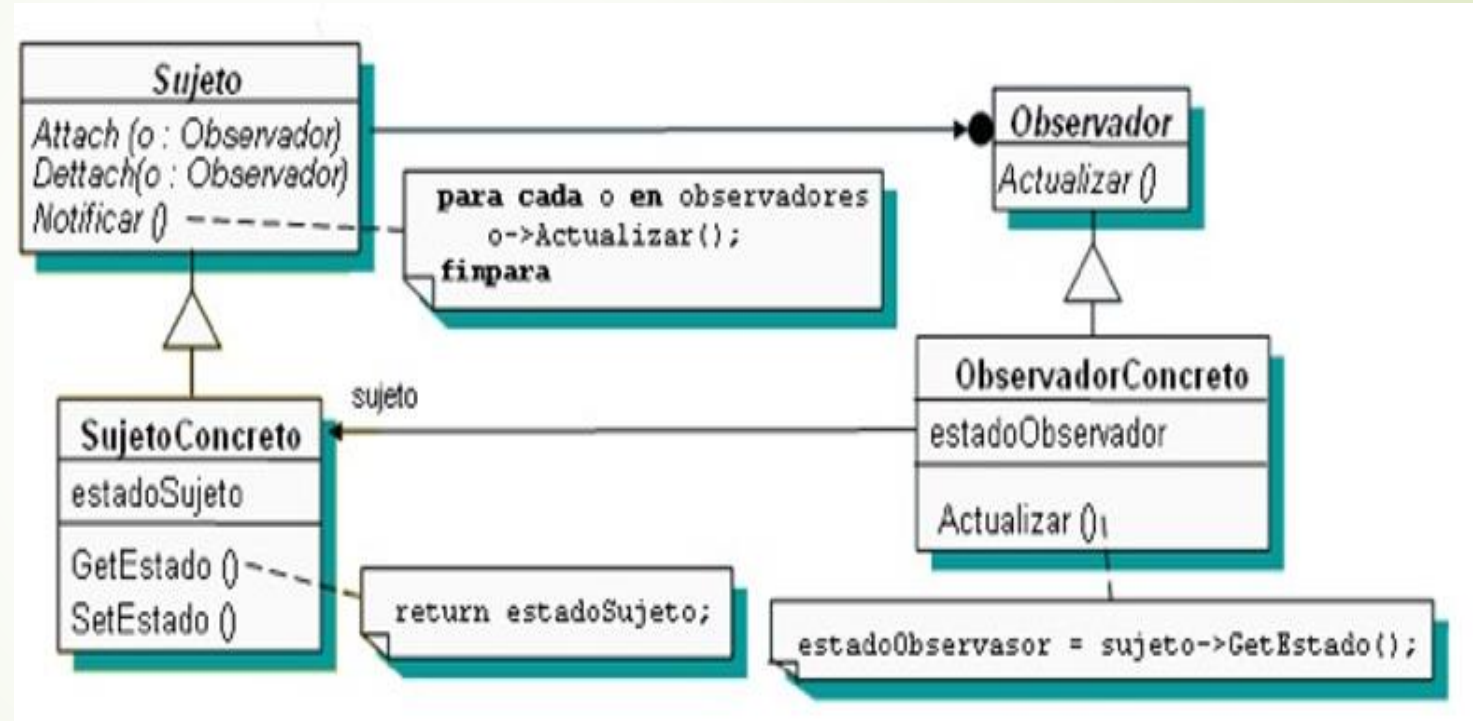
Aplicación

- Escribir las partes fijas de un algoritmo una sola vez y definir las partes que varían en las subclases
- Refactorización de código
- Identificar las diferencias en el código existente
- Separar las diferencias en nuevas operaciones
- Define los puntos de extensión de las subclases



OBSERVADOR

Los objetos son capaces de suscribirse a una serie de eventos que otro objetivo va a emitir, y serán avisados cuando esto ocurra.



Aplicación

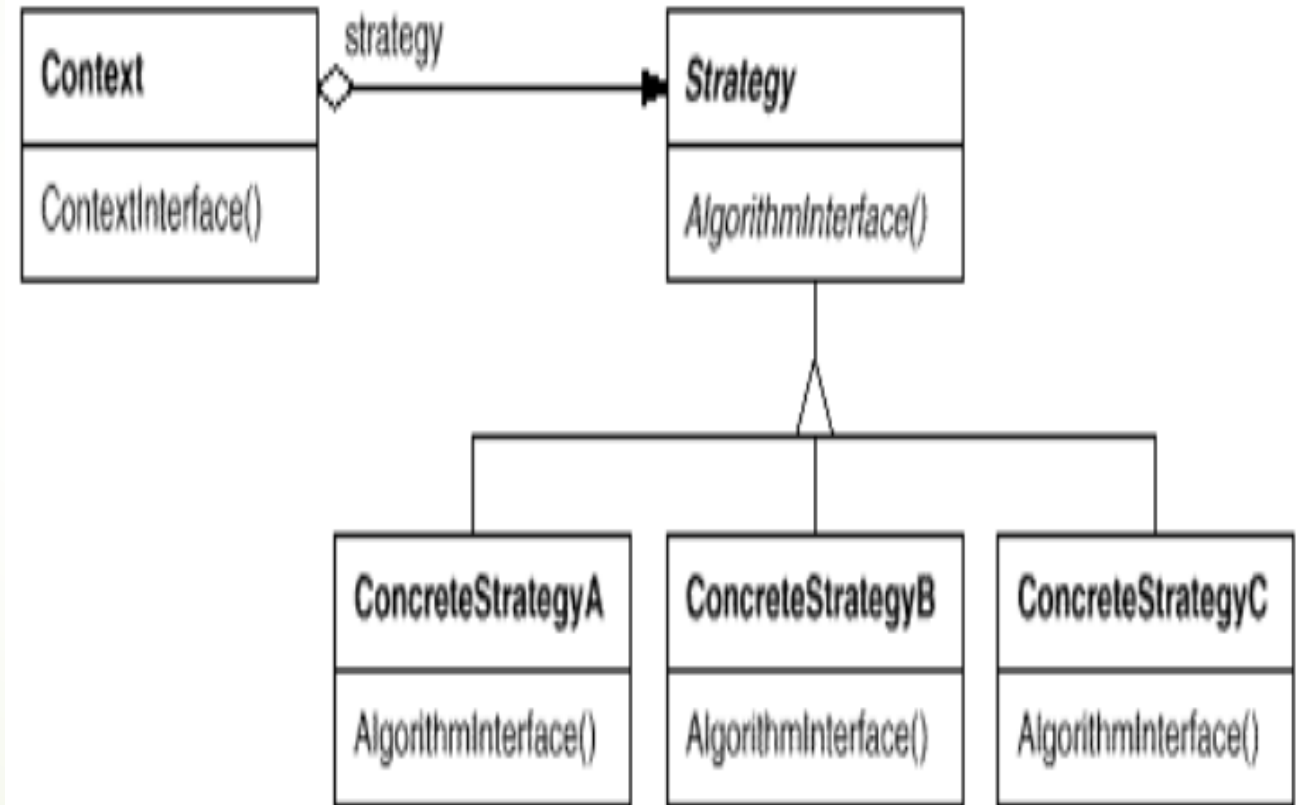
- Un cambio en un objeto requiera cambiar otros y no se sepa cuantos objetos necesitan cambiar
- Un objeto deba ser capaz de notificar a otros sin conocer su clase concreta, evitando así acoplarlos

ESTRATEGIA

Permite la selección del algoritmo que ejecuta cierta acción en tiempo de ejecución.

Aplicación

- Cuando hay muchas clases similares que sólo difieren en algún comportamiento
- Se puede configurar una clase con varios comportamientos
- Se necesitan variantes de un algoritmo
- Para ocultar estructuras de datos complejas que usa el algoritmo
- En vez de utilizar muchas instrucciones condicionales poner los comportamientos alternativos en clases estrategia concretas

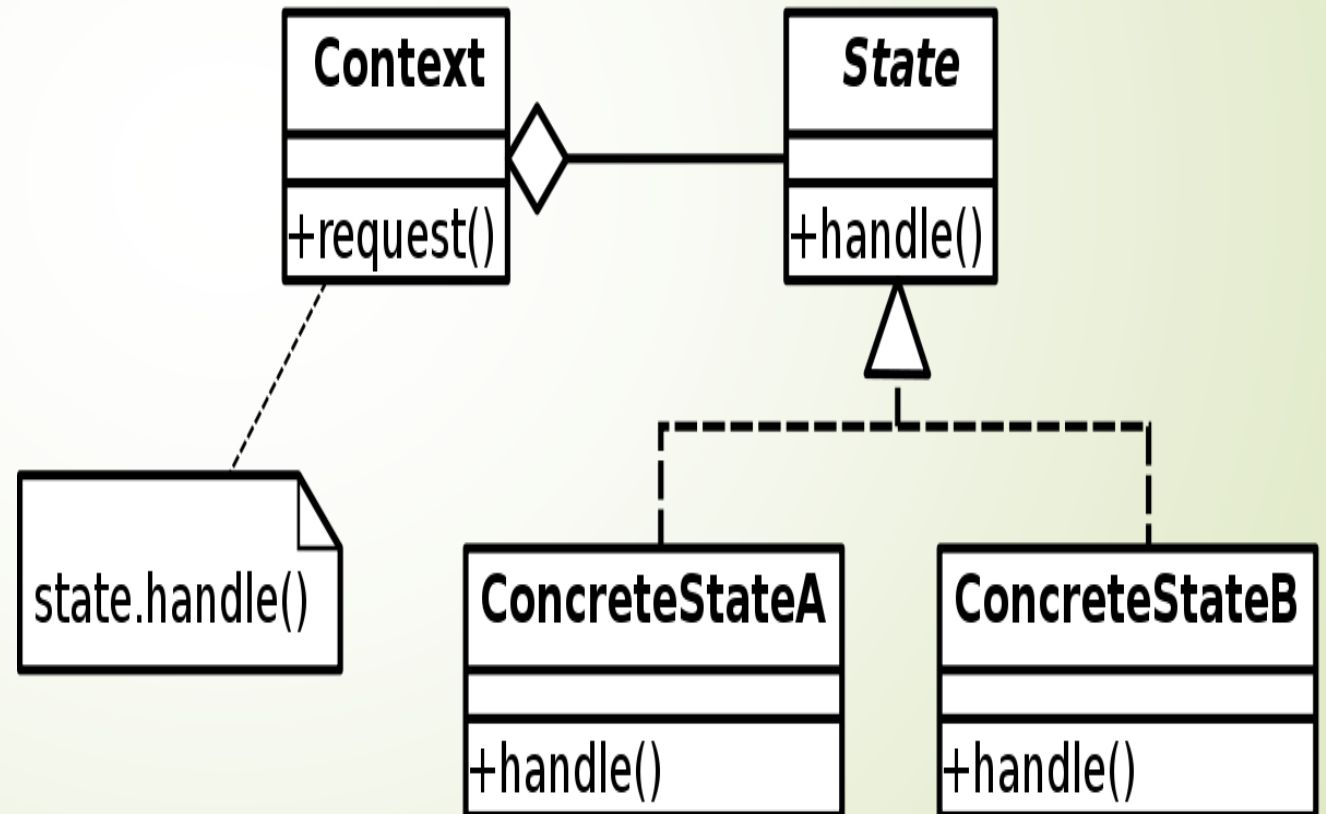


ESTADO

Permite modificar la forma en que un objeto se comporta en tiempo de ejecución, basándose en su estado interno.

Aplicación

- cuando un determinado objeto tiene diferentes estados y también distintas responsabilidades según el estado en que se encuentre en determinado instante
- para simplificar casos en los que se tiene un complicado y extenso código de decisión que depende del estado del objeto



ITERADOR

Se utiliza para poder movernos por los elementos de un conjunto de forma secuencial sin necesidad de exponer su implementación específica.

Aplicación

- Acceder al contenido de un objeto agregado sin revelar su representación interna
- Permitir múltiples tipos de recorridos sobre objetos agregados
- Proporcionar una interfaz uniforme para recorrer diferentes estructuras de agregación (iteración polimórfica)

